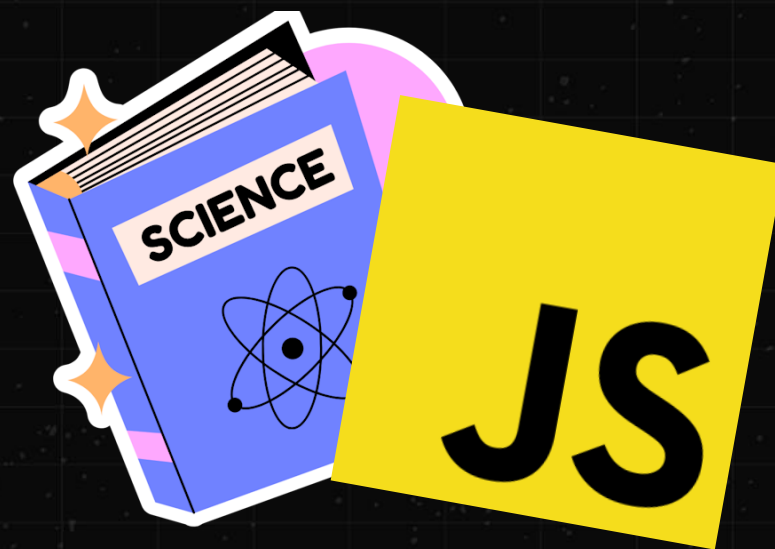


ЛЕКЦИЯ #21

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Продолжаем изучать регулярные выражения
- * Изучаем **возможности группирующих скобок**
- * Подробнее исследуем **жадность квантификаторов** и последствия для производительности

ГРУППИРОВКА

- * Мы уже знаем, что можно использовать символы () для создания «локальной» альтернатиции
- * `Ру(ка | кав(а | ички))`
- * То же самое, что и `Рука | Рукава | Рукавички`
- * А также вместе с квантификаторами
- * `Ру(ка | кав(а | ички)?)`
- * То же самое, что и `Рука | Рукава | Рукавички | Рукав`

НО ЭТО ЕЩЁ НЕ ВСЁ

- ✱ Создавая группу, мы получаем возможность обратиться к ней при обработке результата

```
const regex = /(\d{4})-(\d{2})-(\d{2})/;  
const match = "Сегодня 2025-01-15".match(regex);  
  
console.log(match[0]); // "2025-01-15" (всё совпадение)  
console.log(match[1]); // "2025" (группа 1)  
console.log(match[2]); // "01" (группа 2)  
console.log(match[3]); // "15" (группа 3)
```

РЕФЛЕКСИРУЕМ

- * Каждая скобочная группа **имеет свой порядковый номер**, начиная с **1**
- * Многие **методы, работающие с регулярными выражениями**, позволяют прочитать содержимое групп
- * Например, поэтому **match** **всегда возвращает массив**: нулевой элемент в данном случае — это вся найденная подстрока, а остальные — группы
- * Однако если в **match** передать регулярное выражение с флагом **g**, то **получить доступ к отдельным группам будет нельзя**



```
const text = "Даты: 2025-01-15 и 2025-02-20";

// Без флага g — возвращает первое совпадение + группы
const match = text.match(/(\d{4})-(\d{2})-(\d{2})/);

// [
//   '2025-01-15',
//   '2025',
//   '01',
//   '15',
//   index: 6,
//   input: 'Даты: 2025-01-15 и 2025-02-20',
//   groups: undefined
// ]
console.log(match);

// С флагом g — возвращает все совпадения, но группы теряются
const matchAll = text.match(/(\d{4})-(\d{2})-(\d{2})/g);

// [ '2025-01-15', '2025-02-20' ]
console.log(matchAll);
```


ОБРАТИТЕ ВНИМАНИЕ

- * У всех ответов, за исключением `match` с `g`, в массиве результата есть дополнительные свойства
- * `index` — это индекс UTF-16 юнита, с которого начинается совпадение
- * `input` — это строка, в которой выполнялся поиск
- * `groups` — это свойство, которое мы разберём отдельно сегодня

А КАК **НУМЕРУЮТСЯ** ВЛОЖЕННЫЕ ГРУППЫ?



В ПОРЯДКЕ ОБЪЯВЛЕНИЯ

```
const regex = /((\d{2})(\d{2}))-(\d{2})-(\d{2})/;  
const match = "Сегодня 2025-01-15".match(regex);  
  
console.log(match[0]); // "2025-01-15" (всё совпадение)  
  
console.log(match[1]); // "2025" (группа 1)  
console.log(match[2]); // "20" (группа 2)  
console.log(match[3]); // "25" (группа 3)  
console.log(match[4]); // "01" (группа 4)  
console.log(match[5]); // "15" (группа 5)
```

С АЛЬТЕРНАЦИЯМИ

```
const regex = /((\d{4})|(\d{2}))-(\d{2})-(\d{2})/;  
const match = "Сегодня 2025-01-15".match(regex);  
  
console.log(match[0]); // "2025-01-15" (всё совпадение)  
  
console.log(match[1]); // "2025" (группа 1)  
console.log(match[2]); // "2025" (группа 2)  
console.log(match[3]); // undefined (группа 3)  
console.log(match[4]); // "01" (группа 4)  
console.log(match[5]); // "15" (группа 5)
```

А КАК ЗАПОМИНАЮТСЯ **ГРУППЫ**
С КВАНТИФИКАТОРАМИ?



ТОЛЬКО ПОСЛЕДНЕЕ ВХОЖДЕНИЕ

```
const regex = /((\d{2})+)-(\d{2})-(\d{2})/;  
const match = "Сегодня 2025-01-15".match(regex);  
  
console.log(match[0]); // "2025-01-15" (всё совпадение)  
  
console.log(match[1]); // "2025" (группа 1)  
console.log(match[2]); // "25" (группа 2)  
console.log(match[3]); // "01" (группа 3)  
console.log(match[4]); // "15" (группа 4)
```

ЕСЛИ 0, ТО **undefined**

```
const regex = /(\d{4})?\w+/;  
const match = "Hello World".match(regex);  
  
// [  
//   'Hello',  
//   undefined,  
//   index: 0,  
//   input: 'Hello World',  
//   groups: undefined  
// ]  
console.log(match);
```

КОГДА ГРУППЫ СЛИШКОМ МНОГО

- * Использовать числовые индексы **становится крайне неудобно**
- * К счастью, любую такую **группу можно явно именовать**, чтобы получить осмысленный ассоциативный ключ для доступа к группе
- * По индексу группа **по-прежнему остаётся доступной**
- * Задать имя можно с помощью синтаксиса **?<name>** сразу после (

НАПРИМЕР

```
const regex = /(?!<year>\d{4})-(?!<month>\d{2})-(?!<day>\d{2})/;  
const match = "Сегодня 2025-01-15".match(regex);  
  
console.log(match[0]); // "2025-01-15" (всё совпадение)  
  
console.log(match[1]); // "2025" (группа 1)  
console.log(match.groups.year); // "2025" (группа 1)  
console.log(match.groups.month); // "01" (группа 2)  
console.log(match.groups.day); // "15" (группа 3)
```

ТЕПЕРЬ СТАЛ ЯСЕН СМЫСЛ **groups**

- * Это просто словарь именованных групп
- * Разумеется, не может быть двух и более групп с одним именем
- * По крайней мере, они не должны пересекаться (быть в разных альтернативах)
- * Например, `/(?<year>\d{4})|(?<year>\d{2})/` допустимо в ES2025

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВСЁ НЕ ТАК СТРАШНО

- * С регулярными выражениями **нужна практика**
- * И работа с группами тут не исключение

А С ЗАМЕНОЙ КАК?





```
const regex = /(?!<year>\d{4})-(?!<month>\d{2})-(?!<day>\d{2})/;

// Сегодня 15-01-2025
console.log("Сегодня 2025-01-15".replace(regex, "$3-$2-$1"));

// Сегодня 15-01-2025
console.log("Сегодня 2025-01-15".replace(regex, (substr, year, month, day) => `${day}-${month}-${year}`));

// Сегодня 15-01-2025
console.log("Сегодня 2025-01-15".replace(regex, "$<day>-$<month>-$<year>"));

// Сегодня 15-01-2025
console.log("Сегодня 2025-01-15".replace(regex, (...args) => {
  const { year, month, day } = args.at(-1);
  return `${day}-${month}-${year}`;
}));
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ

- * Методы строки `replace` и `replaceAll` могут принимать два типа параметров: строку или функцию
- * Внутри строки допускается **использовать специальные переменные**
- * Например, `$1` для замены на первую группу или `$<name>` для замены на группу по имени
- * Вариант с функцией **принимает все эти параметры как аргументы**

ДРУГИЕ СТРОКОВЫЕ ПЕРЕМЕННЫЕ



"\$\$"	// Вставляет символ \$	\$\$ → "\$"
"\$&"	// Вся найденная подстрока (match)	\$& → "2025-01-15"
"\$`"	// Весь текст **до** совпадения	\$` → "Сегодня "
"\$'"	// Весь текст **после** совпадения	\$' → " и завтра"
"\$n"	// Группа по номеру (n = 1–99)	\$1, \$2, \$3
"\$<name>"	// Именованная группа	\$<year>, \$<month>, \$<day>

```
/**
 * @param match - Вся найденная подстрока (совпадение целиком)
 * @param groups - Захваченные группы по порядку (каждая группа — отдельный аргумент)
 * @param offset - Позиция начала совпадения в строке (в UTF-16 единицах)
 * @param string - Исходная строка (целиком)
 * @param groupsObject - Объект с именованными группами (только если есть `(<name>...)`)
 * @returns Строка, которая заменит найденное совпадение
 */
function replacer(
  match: string,
  ...args: [...string[], number, string, Record<string, string>?]
): string {
  return "Строка заменитель";
}
```

ССЫЛКА НА ГРУППУ В САМОМ ВЫРАЖЕНИИ

- * Удивительно, но мы **можем обратиться к группе по её индексу или имени** прямо из самого регулярного выражения
- * Это позволяет создавать условия, **когда последовательность должна быть такой же, как и в группе**
- * Например, для проверки баланса кавычек



```
// По индексу — \1, \2, \3, ...  
const regex1 = /(["']).*?\1/; // Находит строку в кавычках  
console.log(regex1.exec("Hello")); // ["Hello", ""]  
console.log(regex1.exec("Hello")); // ["Hello", ""]  
console.log(regex1.exec("Hello\"")); // null  
  
// По имени — \k<name>  
const regex2 = /(?!<quote>["']).*?\k<quote>/;  
console.log(regex2.exec("Hello")); // ["Hello", ""]  
console.log(regex2.exec("Hello")); // ["Hello", ""]  
console.log(regex2.exec("Hello\"")); // null
```

ЭТО ОЧЕНЬ МОЩНЫЙ МЕХАНИЗМ

- ✱ Который позволяет описывать более богатые грамматики
- ✱ А также оптимизировать работу движка регулярных выражений

ГРУППА БЕЗ ЗАПОМИНАНИЯ

- * Иногда группа нам нужна **только для того, чтобы использовать альтернатию**
- * Или **применить квантификатор к ней** (или и то и другое)
- * Движок делает лишнюю работу по запоминанию, а **самых ссылок становится больше**
- * Чтобы исправить это, можно явно разметить группу как группу без запоминания (незахватывающую)
- * Задать такую группу можно с помощью синтаксиса **?:** сразу после (



// ❌ Плохо: группа захватывает, но не используется

```
const regex1 = /(https?:\V\V)?(www\.)?/;
```

// `www.` — захватывается, но не нужна

// ✅ Хорошо: явно указываем, что группа не нужна

```
const regex2 = /(https?:\V\V)?(?:www\.)?/;
```

// Для альтернации

```
const regex3 = /Py(?:ка | кав(?:а | ички)?)/;
```

МОЖНО КОМБИНИРОВАТЬ



```
const regex = /(\d{4})-((?:\d{2}(?:-|$))+)/;  
const match = regex.exec("2025-01-15");
```

```
console.log(match[0]); // "2025-01-15"  
console.log(match[1]); // "2025"  
console.log(match[2]); // "01-15"
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЖКА, СТОЖКААА!

ВЫДЫХАЕМ

- * Просто держите в голове, что если вам не нужна ссылка на группу, то **её можно не создавать**
- * Это особенно актуально, когда групп много и **некоторые из них вы хотите запоминать, а другие – нет**

КВАНТИФИКАТОР НАД КВАНТИФИКАТОРОМ

- * Логично, что раз мы можем сделать квантификатор для символа или группы, то **можно и комбинировать их**
- * Проблема такого подхода в том, что можно очень легко создать регулярное выражение, которое **будет работать крайне неэффективно**



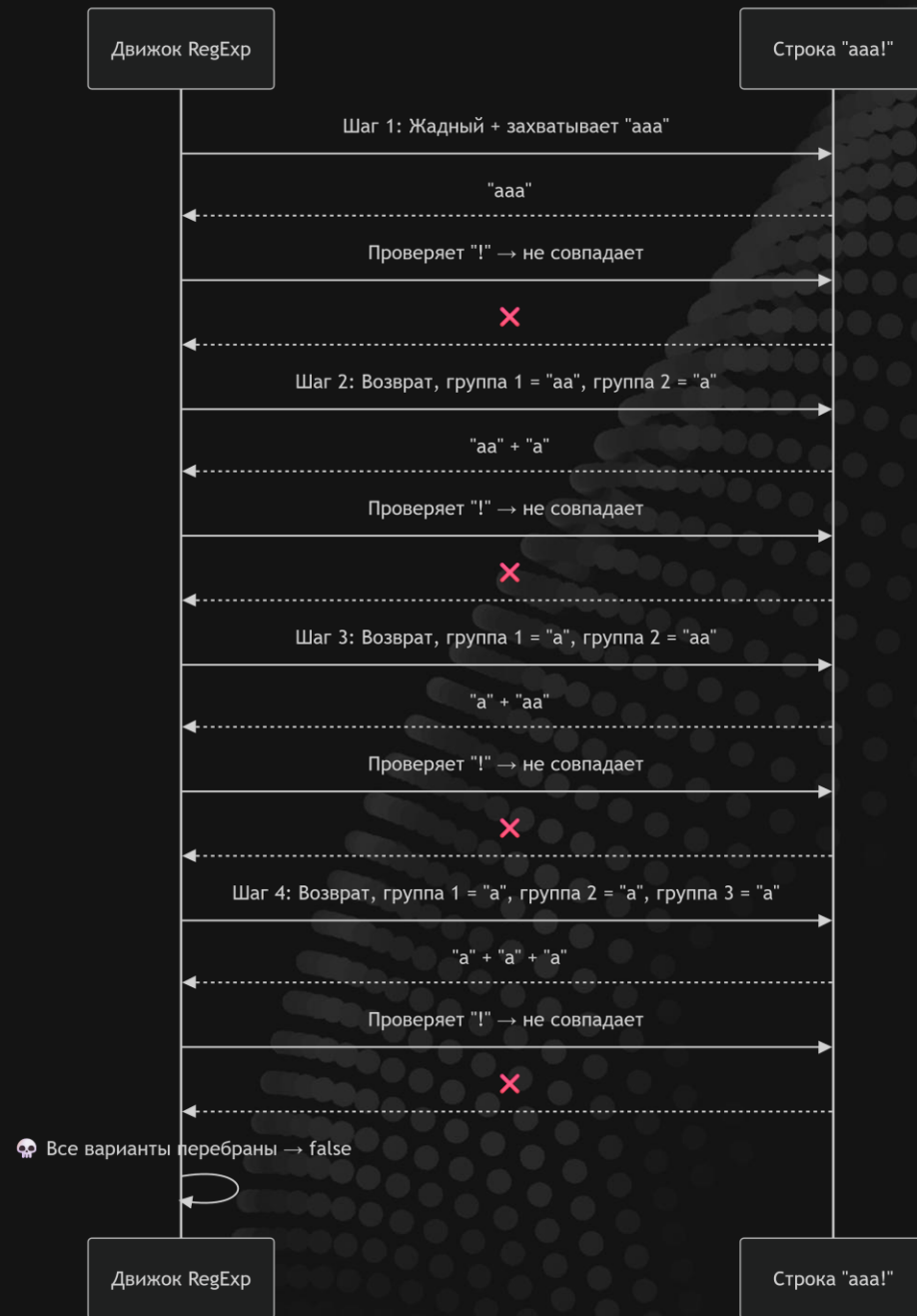
```
// ❌ Опасно: экспоненциальное время работы  
const regex = /^[a-z+]*$/; // Повторение группы с квантификаторами внутри  
  
// Может привести к catastrophic backtracking (катастрофические возвраты)  
const text = "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa!";  
const start = performance.now();  
const result = regex.test(text);  
console.log(performance.now() - start); // Может зависнуть!
```

ПРОБЛЕМА В ВОЗВРАТАХ

- * Жадный квантификатор `+` пытается найти максимальное количество символов `[a-z]` и в итоге доходит до `!`
- * Далее вступает в игру `*$`, что означает, что после `([a-z]+)` должен быть конец строки, однако `*` допускает, что таких групп может быть от 0 до бесконечности
- * Движок начинает перебирать все возможные варианты разбиения строки на группы, чтобы найти совпадение

НАПРИМЕР

- ✱ Для строки из 30 символов он попыбует разбить её на 1 группу из 30 символов, затем на 2 группы (29+1, 28+2, ...), затем на 3 группы и так далее
- ✱ Это приводит к экспоненциальному росту числа проверок – катастрофическому возврату (catastrophic backtracking)



ЗДЕСЬ ДЕЛО В «ЖАДНОСТИ»?



НЕТ, ДЕЛО НЕ В ЭТОМ

- * Жадность определяет, **что считать успешным результатом и когда делать возврат**
- * Например, нежадный (ленивый) поиск будет делать возврат при первом успешном совпадении, **чтобы проверить, что условие выполнено минимально**
- * Жадный же **потребляет все допустимые символы до первого несовпадения** и уже потом делает возврат

НАПРИМЕР



```
const text = "<h1>Заголовок</h1>";

// Жадный — захватывает всё до последнего >
const greedy = /<.+>/.exec(text);
console.log(greedy[0]); // "<h1>Заголовок</h1>"

// Ленивый — захватывает минимально возможное
const lazy = /<.+?>/.exec(text);
console.log(lazy[0]); // "<h1>"
```

ПАТТЕРН «АТОМАРНАЯ ГРУППА»



```
// ✅ С атомарной группой — безопасно (фиксируем захват)  
// Атомарная группа запрещает возврат внутри ([a-z]+)  
const regex = /^[a-z]+\1*$/;  
  
const text = "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa!";  
const start = performance.now();  
const result = regex.test(text);  
console.log(performance.now() - start); // Работает быстро
```

РЕФЛЕКСИРУЕМ

- ✱ Мы применили квантификатор ***** не к группе, а к тому, что она захватила (через обратную ссылку **\1**)
- ✱ Это **свело количество возвратов до одного**:
либо нашлось совпадение с захваченной группой, либо нет

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Просто держите в голове, что если вы **применяете квантификатор к группе, внутри которой уже есть квантификатор**, то тут нужно быть очень внимательным
- * Не обязательно сразу бежать заменять код на обратные ссылки — **иногда вложенное условие само по себе исключает такие ситуации**
- * Но помните про опасность: неаккуратная регулярка может стать причиной зависания браузера или сервера
- * Можно настроить линтер, например, [eslint-plugin-regexp](#)

РЕГУЛЯРНЫЕ ВЫРАЖЕНИЯ И АВТОМАТЫ

- * Движок регулярных выражений компилирует переданное выражение и строит некоторый автомат, который уже и будет осуществлять поиск
- * В зависимости от реализации и выражения, автомат может быть детерминированным (быстрее) или недетерминированным (мощнее)
- * Для визуализации автомата и отладки есть много полезных сервисов
- * Например, [debuggex](#) или [regex101](#)

ПОЧЕМУ ДКА
БЫСТРЕЕ?

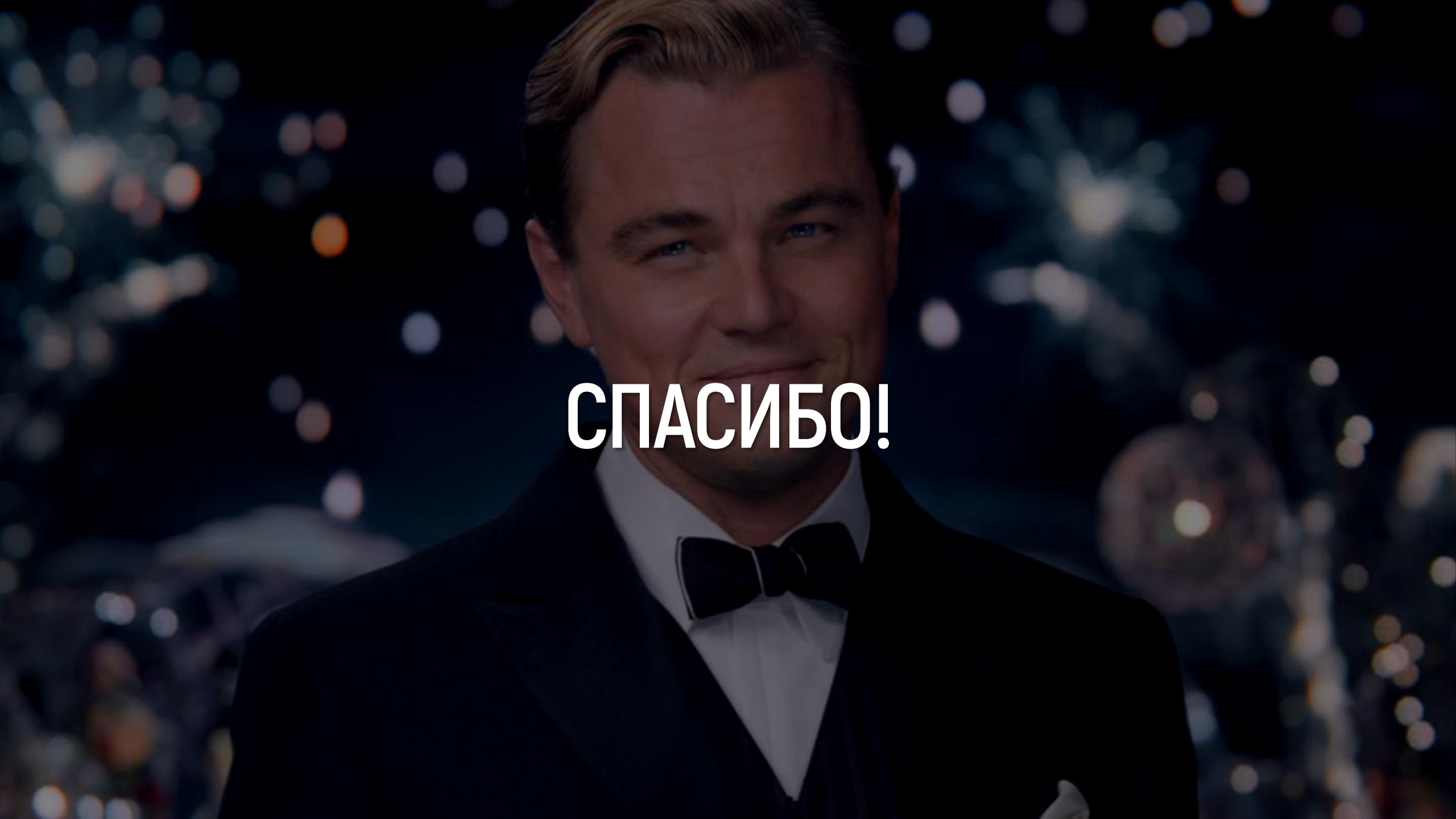


ДЕТЕРМИНИРОВАННЫЕ ПЕРЕХОДЫ

- * ДКА не нуждается в таком количестве возвратов, как НКА, поэтому операция поиска выполняет куда меньше работы, что выливается в более высокую эффективность
- * С другой стороны, **построение самого ДКА** может быть более трудоёмкой операцией, и такой автомат зачастую будет менее «могущественным»
- * НКА же, напротив, **проще построить, а его возможности зачастую выше**
- * В современных реализациях JavaScript используются НКА со множественными оптимизациями

ПОДВЕДЁМ ИТОГИ

- * Группирующие скобки куда мощнее, чем кажутся на первый взгляд — они **позволяют ссылаться на найденные фрагменты**
- * Ссылаться можно как в результатах поиска, так и **внутри самого регулярного выражения**
- * Это открывает возможности **для описания более выразительных грамматик**
- * При написании регулярных выражений нужно быть аккуратными со вложенными квантификаторами, ведь они могут **привести к проблемам с производительностью**
- * Такое регулярное выражение может запросто «повесить» сервер

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!